



Funciones para conjuntar información en pandas

`concat()`

Imagina que en tu empresa los registros de proveedores están en archivos separados; sin embargo, todos cuentan con la misma estructura. ¿Cómo integrarías la información de dichos proveedores?

La función `concat()` es una de las funciones de pandas más utilizadas para integrar información de diferentes fuentes. Para empezar a familiarizarnos, veamos un ejemplo y recordemos el trabajo con DataFrames.

Supongamos que tenemos tres DataFrames definidos de la siguiente manera:

```
df1 = pd.DataFrame(  
    {  
        "A": ["A0", "A1", "A2", "A3"],  
        "B": ["B0", "B1", "B2", "B3"],  
        "C": ["C0", "C1", "C2", "C3"],  
        "D": ["D0", "D1", "D2", "D3"],  
    },  
    index=[0, 1, 2, 3],  
)
```

```
df2 = pd.DataFrame(  
    {  
        "A": ["A4", "A5", "A6", "A7"],  
        "B": ["B4", "B5", "B6", "B7"],  
        "C": ["C4", "C5", "C6", "C7"],  
        "D": ["D4", "D5", "D6", "D7"],  
    },  
    index=[4, 5, 6, 7],  
)
```

```
df3 = pd.DataFrame(  
    {  
        "A": ["A8", "A9", "A10", "A11"],  
        "B": ["B8", "B9", "B10", "B11"],  
        "C": ["C8", "C9", "C10", "C11"],  
        "D": ["D8", "D9", "D10", "D11"],  
    },  
    index=[8, 9, 10, 11],  
)
```

Observa que los tres DataFrames comparten la misma estructura. Piénsala como información proveniente de tres fuentes distintas, pero organizadas de manera uniforme. Todos tienen las mismas columnas e incluso fueron creados de forma que sus índices continúan secuencialmente uno tras otro.

df1

<

Figura 1. Estructura de los DataFrames.

La función `concat()` de **pandas** **toma una lista o un diccionario de objetos de tipo homogéneos y los concatena**. Esta función tiene varios parámetros con los que puedes experimentar para obtener los resultados deseados. Recuerda que, para más información, siempre puedes consultar la documentación oficial, este enlace lo encontrarás en la sección de Material adicional de este recurso.

Para nuestro ejemplo, lo que queremos es concatenar la información de los tres DataFrames, esto lo puedes hacer con la siguiente instrucción:

```
result = pd.concat([df1, df2, df3])
```

y te da como resultado lo siguiente:

```
result = pd.concat([df1, df2, df3])
result
```

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A4	B4	C4	D4
5	A5	B5	C5	D5
6	A6	B6	C6	D6
7	A7	B7	C7	D7
8	A8	B8	C8	D8
9	A9	B9	C9	D9
10	A10	B10	C10	D10
11	A11	B11	C11	D11

Figura 2. DataFrames concatenados.

En este caso, funcionó bien debido a que los índices eran consecutivos. Sin embargo, es probable que cada parte que quieras concatenar tenga sus propios índices; en ese caso, al momento de concatenar, obtendríamos algo como esto:

```
result = pd.concat([df1, df2, df3])
result
```

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
0	A4	B4	C4	D4
1	A5	B5	C5	D5
2	A6	B6	C6	D6
3	A7	B7	C7	D7
0	A8	B8	C8	D8
1	A9	B9	C9	D9
2	A10	B10	C10	D10
3	A11	B11	C11	D11

Figura 3. DataFrames concatenados con índices diferentes.

Esto lo puedes arreglar utilizando la función `reset_index`, con los parámetros `drop= True` (que elimina el índice actual), `inplace= True` (que modifica el dataframe), quedando el DataFrame de la siguiente manera:

```
result.reset_index(drop= True, inplace= True)
result
```

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A4	B4	C4	D4
5	A5	B5	C5	D5
6	A6	B6	C6	D6
7	A7	B7	C7	D7
8	A8	B8	C8	D8
9	A9	B9	C9	D9
10	A10	B10	C10	D10
11	A11	B11	C11	D11

Figura 4. Arreglo del DataFrame usando la función `reset_index`.

Otra opción es que, al momento de hacer la concatenación, utilices el parámetro `ignore_index=True`

```
result = pd.concat([df1, df2, df3], ignore_index=True)
```

Este tipo de concatenación está hecha de manera **vertical**, es decir, un DataFrame debajo de otro. Sin embargo, podemos encontrar situaciones en las que preferiríamos concatenar la información de DataFrames de otras formas.

Veamos cómo realizar algunas operaciones de concatenación del álgebra relacional usando la función `concat()`.

Tomemos en cuenta el `df1` definido anteriormente y definamos ahora un `df4` de la siguiente manera:

```
df4 = pd.DataFrame(  
    {  
        "B": ["B2", "B3", "B6", "B7"],  
        "D": ["D2", "D3", "D6", "D7"],  
        "F": ["F2", "F3", "F6", "F7"],  
    },  
    index=[2, 3, 6, 7],  
)
```

tenemos entonces, `df1` y `df4` de la siguiente forma:

df1					df4			
	A	B	C	D		B	D	F
0	A0	B0	C0	D0	2	B2	D2	F2
1	A1	B1	C1	D1	3	B3	D3	F3
2	A2	B2	C2	D2	6	B6	D6	F6
3	A3	B3	C3	D3	7	B7	D7	F7

Figura 5. Definición de DataFrames `df1` y `df4`.

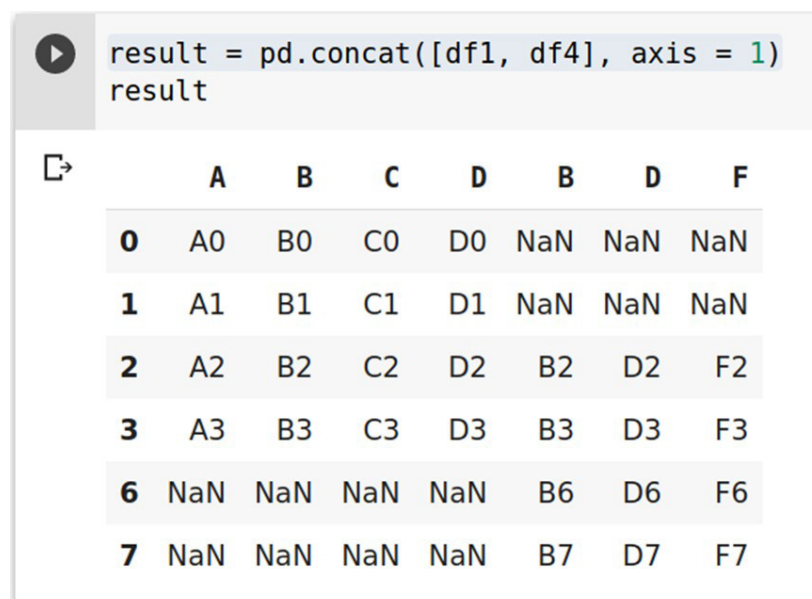
Podemos considerar, en este caso, que los índices de ambos DataFrames hacen referencia al mismo objeto. Entonces tenemos como índices comunes las filas con índice 2 y 3.

Si quieres concatenar la información de ambos DataFrames sin perder información, puedes hacer una reunión externa, es decir, un `outer join`. Esta es la operación predeterminada cuando utilizamos la función `concat()` con el parámetro `axis=1` para que efectúe la concatenación de un DataFrame a la derecha del otro. Esto podemos verlo como una **concatenación horizontal**. Si omitiéramos este parámetro, el *default* es que `concat()` concatene un DataFrame debajo de otro.

Para hacer la concatenación horizontal, la operación quedaría de la siguiente forma:

```
result = pd.concat([df1, df4], axis = 1)
```

dejando el resultado así:



```
result = pd.concat([df1, df4], axis = 1)
result
```

	A	B	C	D	B	D	F
0	A0	B0	C0	D0	NaN	NaN	NaN
1	A1	B1	C1	D1	NaN	NaN	NaN
2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	B3	D3	F3
6	NaN	NaN	NaN	NaN	B6	D6	F6
7	NaN	NaN	NaN	NaN	B7	D7	F7

Figura 6. Concatenación horizontal.

Algo que debes tomar en cuenta es que, como tenemos nombres de columnas con los mismos valores, **pandas conserva ambas**, considerando que las primeras corresponden al primer DataFrame de la operación y las de la derecha, al DataFrame que se encuentra a la derecha en la operación.

En este caso, podría ser **útil renombrar las columnas** de nuestro DataFrame resultante para evitar ambigüedades. Esto lo puedes hacer con la función `set_axis()`, pasando como parámetros la lista de las nuevas etiquetas para las columnas, `axis=1` para denotar que se trata de los índices de las columnas, y `inplace=True` para indicar que deseas modificar el DataFrame directamente.

La instrucción quedaría de la siguiente forma:

```
result.set_axis(['A1', 'B1', 'C1', 'D1', 'B2', 'D2', 'F2'],  
axis=1, inplace=True)
```

Dando como resultado:

```
▶ result.set_axis(['A1', 'B1', 'C1', 'D1', 'B2', 'D2', 'F2'], axis=1, inplace=True)  
result
```

	A1	B1	C1	D1	B2	D2	F2
0	A0	B0	C0	D0	NaN	NaN	NaN
1	A1	B1	C1	D1	NaN	NaN	NaN
2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	B3	D3	F3
6	NaN	NaN	NaN	NaN	B6	D6	F6
7	NaN	NaN	NaN	NaN	B7	D7	F7

Figura 7. Columnas renombradas con la función `set_axis()`.

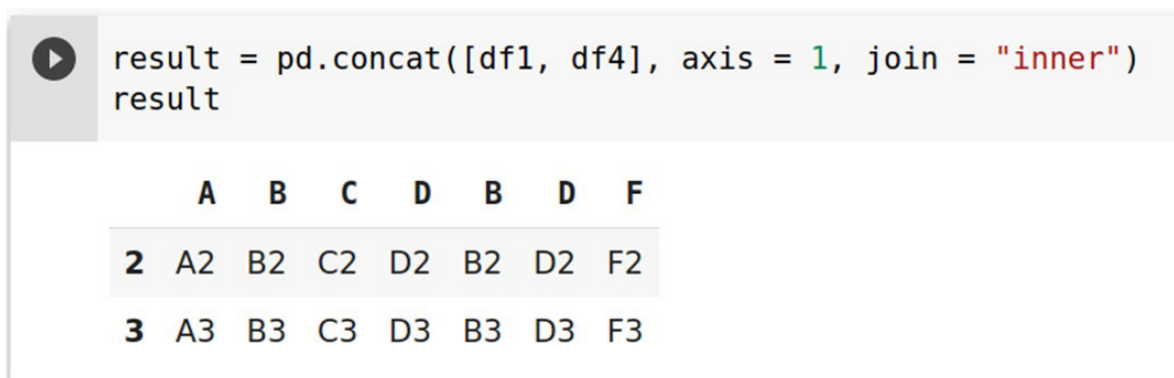
Ahora que renombramos las columnas para no tener nombres repetidos, vamos a analizar el resultado. Todas las filas de los DataFrames se conservaron, y las que tenían índices en común formaron una nueva fila con valores para todas las columnas. Las que no coincidían se complementaron con el valor NaN. El valor NaN significa "Not a Number" y se utiliza para indicar que el dato no está disponible o que no existe.

La operación que acabamos de realizar puede tener sentido en algunas situaciones, especialmente cuando deseas conservar la información contenida en ambos DataFrames. Sin embargo, es posible que en ciertos casos sólo te interese conservar las filas cuyo índice esté presente en ambos DataFrames. Es aquí donde puedes utilizar una **reunión interna**, o `inner join`. Esta operación conservará únicamente los renglones cuyo índice esté en ambos DataFrames.

La operación quedaría de la siguiente forma:

```
result = pd.concat([df1, df4], axis = 1, join = "inner")
```

El resultado es el siguiente:



	A	B	C	D	B	D	F
2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	B3	D3	F3

Figura 8. Función `inner join` aplicada al DataFrame.

Como puedes ver, conservamos solamente las filas con los índices 2 y 3.

Si deseas conservar todas las filas de alguno de los dos DataFrames, puedes aplicar la función `reindex()`, pasando como parámetro el índice del DataFrame que deseas conservar.

Por ejemplo, si quisiéramos hacer un `left join` (conservar todos los registros del DataFrame de la izquierda), ejecutamos la siguiente instrucción:

```
result = pd.concat([df1, df4], axis = 1).reindex(df1.index)
```

nos da el siguiente resultado:



```
result = pd.concat([df1, df4], axis = 1).reindex(df1.index)
result
```

	A	B	C	D	B	D	F
0	A0	B0	C0	D0	NaN	NaN	NaN
1	A1	B1	C1	D1	NaN	NaN	NaN
2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	B3	D3	F3

Figura 9. Función `left join`.

Y si quisiéramos hacer un `right join` (conservar todos los registros del DataFrame de la derecha), ejecutaríamos la instrucción:

```
result = pd.concat([df1, df4], axis = 1).reindex(df4.index)
```

que nos da el siguiente resultado:

```
result = pd.concat([df1, df4], axis = 1).reindex(df4.index)  
result
```

	A	B	C	D	B	D	F
2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	B3	D3	F3
6	NaN	NaN	NaN	NaN	B6	D6	F6
7	NaN	NaN	NaN	NaN	B7	D7	F7

Figura 10. Función `right join`.

Recordemos que estas operaciones de concatenación horizontal se realizan de acuerdo con el índice de los DataFrames con los que estemos trabajando.

Si tuviéramos dos DataFrames con el mismo número de índices (es decir, el mismo número de renglones) y con los índices en el mismo orden, la operación de concatenación horizontal funcionaría como si “pegáramos” los dos DataFrames.

Antes de continuar...

Hasta este momento, te hemos presentado la función `concat`, que tiene varias formas de operar para combinar información proveniente de diferentes fuentes, además de ejemplos que esperamos te ayuden a tener un mejor entendimiento de su uso.

Ahora, pensando en la información que manejas en tu lugar de trabajo:

- ¿Cómo podrías utilizar estas funciones para crear DataFrames que representen información de tu entorno laboral, en donde se concentre información de varias fuentes para poder analizarla de forma integral?
- ¿Cuál o cuáles de las diferentes formas de usar la función `concat` crees que te podrían servir más para trabajar con la información que manejas día a día?

| Funciones join y merge

merge ()

Imagina una franquicia de talleres mecánicos de servicio rápido, donde se tienen datos de clientes, entre ellos los servicios que han realizado a su automóvil. Los datos de los clientes están asociados a una sucursal, aunque dichos clientes pueden solicitar sus servicios en otra. Además, se cuenta con las diferentes facturas de los clientes. ¿Podrías unir a los clientes con las facturas para tener una única tabla?, ¿podrías buscar a los clientes que no han realizado ningún servicio en cierto periodo?, ¿qué funciones de pandas te podrían servir?

pandas cuenta con operaciones de reunión de alto desempeño. Por ejemplo, el método `merge()` puede utilizarse para llevar a cabo las operaciones de reunión del álgebra relacional (**JOINS**), tanto en Series como en DataFrames. Esta función tiene varios parámetros que pueden ajustarse para obtener los resultados deseados.

Aquí te mostraremos algunos ejemplos con las configuraciones más comunes. Recuerda que puedes consultar la documentación oficial en la sección de Material adicional de esta lectura.

Para este ejemplo, trabajaremos con dos DataFrames a los que llamaremos `left` y `right`.

```
left = pd.DataFrame(  
    {  
        "key": ["K0", "K1", "K2", "K3"],  
        "A": ["A0", "A1", "A2", "A3"],  
        "B": ["B0", "B1", "B2", "B3"],  
    }  
)
```

```
right = pd.DataFrame(
    {
        "key": ["K0", "K1", "K2", "K3"],
        "C": ["C0", "C1", "C2", "C3"],
        "D": ["D0", "D1", "D2", "D3"],
    }
)
```

left

	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2
3	K3	A3	B3

right

	key	C	D
0	K0	C0	D0
1	K1	C1	D1
2	K2	C2	D2
3	K3	C3	D3

Figura 11. DataFrames `left` y `right`.

Observa que, en estos dos DataFrames, hay una columna común llamada **"key"**. Vamos a asumir que los DataFrames están relacionados por medio de esta columna.

Utilizando `merge()` podemos relacionar la información contenida en estos dos DataFrames, pasando como parámetros ambos DataFrames y la llave o columna que debe coincidir para hacer la relación:

```
result = pd.merge(left, right, on="key")
```

```
▶ result = pd.merge(left, right, on="key")
result
```

	key	A	B	C	D
0	K0	A0	B0	C0	D0
1	K1	A1	B1	C1	D1
2	K2	A2	B2	C2	D2
3	K3	A3	B3	C3	D3

Figura 12. Relación de los dos DataFrames utilizando la función `merge()`.

Cabe mencionar que la función `merge` puede llamarse de dos formas:

La primera, como se utilizó en el ejemplo anterior:

```
python
CopiarEditar
pd.merge(dataFrame1, dataFrame2, ...)
```

Y también:

```
python
CopiarEditar
dataFrame1.merge(dataFrame2, ...)
```

Donde los tres puntos (...) indican que podemos pasar más argumentos a la función.

Para este ejemplo específico, podemos utilizar las siguientes instrucciones, que nos darán todas el mismo resultado:

```
result = pd.merge(left, right, on="key")
```

Ejecutamos la función `merge` de pandas, pasando como parámetro los DataFrames `left` y `right`, y especificando que la columna por medio de la cual queremos hacer la relación entre ambos DataFrames es `key`. Ambos DataFrames tienen una columna llamada `key`.

```
result = pd.merge(left, right)
```

Ejecutamos la función `merge` de pandas, pasando como parámetro los DataFrames `left` y `right`. En este caso, no especificamos cuál es la columna que queremos utilizar para hacer la relación, entonces pandas busca automáticamente las columnas comunes. En este ejemplo, únicamente la columna `key` es común entre ambos DataFrames.

```
python
```

```
CopiarEditar
```

```
result = left.merge(right, left_on='key', right_on='key')
```

Aquí ejecutamos la función `merge` sobre el DataFrame `left`, pasando como parámetro el DataFrame `right`. Especificamos cuál es la columna del DataFrame de la izquierda que queremos relacionar (`left_on='key'`) y cuál es la columna del DataFrame de la derecha que queremos relacionar (`right_on='key'`).

```
python
```

```
CopiarEditar
```

```
result = left.merge(right)
```

En este caso, también ejecutamos la función `merge` sobre el DataFrame `left`, pasando como parámetro el DataFrame `right`. Como no especificamos cuáles son las columnas que queremos utilizar para hacer la relación, pandas busca las columnas comunes. Nuevamente, sólo la columna `key` es la columna común entre ambos DataFrames.

El resultado generado es:

	key	A	B	C	D
0	K0	A0	B0	C0	D0
1	K1	A1	B1	C1	D1
2	K2	A2	B2	C2	D2
3	K3	A3	B3	C3	D3

	key	A	B	C	D
0	K0	A0	B0	C0	D0
1	K1	A1	B1	C1	D1
2	K2	A2	B2	C2	D2
3	K3	A3	B3	C3	D3

Figura 13. Resultado de la operación `merge()` con columna común `key`.

Veamos ahora un ejemplo más interesante, con 2 llaves, redefinamos nuestros DataFrames `left` y `right` de la siguiente manera:

```
left = pd.DataFrame(  
    {  
        "key1": ["K0", "K0", "K1", "K2"],  
        "key2": ["K0", "K1", "K0", "K1"],  
        "A": ["A0", "A1", "A2", "A3"],  
        "B": ["B0", "B1", "B2", "B3"],  
    }  
)
```



```
right = pd.DataFrame(
    {
        "key1": ["K0", "K1", "K1", "K2"],
        "key2": ["K0", "K0", "K0", "K0"],
        "C": ["C0", "C1", "C2", "C3"],
        "D": ["D0", "D1", "D2", "D3"],
    }
)
```

left

	key1	key2	A	B
0	K0	K0	A0	B0
1	K0	K1	A1	B1
2	K1	K0	A2	B2
3	K2	K1	A3	B3

right

	key1	key2	C	D
0	K0	K0	C0	D0
1	K1	K0	C1	D1
2	K1	K0	C2	D2
3	K2	K0	C3	D3

Figura 14. Redefinición de los DataFrames `left` y `right`.

Ahora hagamos el `merge()` pero de las dos llaves, **donde deben coincidir la combinación de valores**:

```
result = pd.merge(left, right, on=["key1", "key2"])
```

	key1	key2	A	B	C	D
0	K0	K0	A0	B0	C0	D0
1	K1	K0	A2	B2	C1	D1
2	K1	K0	A2	B2	C2	D2

Figura 15. Función `merge()` de las dos llaves.

En la función `merge`, el argumento `how` especifica qué valores se incluirán en el resultado, permitiéndonos así implementar las operaciones `JOIN` del álgebra relacional y la operación de producto cartesiano. Si no se especifica el parámetro `how`, el valor *default* es "inner".

Método	Operación	Descripción
left	LEFT JOIN	Utiliza todos los valores del DataFrame izquierdo, aunque no tengan relación con el DataFrame derecho.
right	RIGHT JOIN	Utiliza todos los valores del DataFrame derecho, aunque no tengan relación con el DataFrame izquierdo.
outer	OUTER JOIN	Utiliza todos los valores del DataFrame izquierdo y todos los valores del DataFrame derecho.
inner	INNER JOIN	Utiliza sólo los valores del DataFrame izquierdo que estén relacionados con valores del DataFrame derecho.
cross	CROSS JOIN	Crea el producto cartesiano de todos los valores del DataFrame izquierdo con todos los valores del DataFrame derecho.

Tabla 1. Operaciones `JOIN` con la función `merge()`.

Veamos cómo quedaría el resultado de aplicar la función `merge`, con diferentes valores para el parámetro `how` en este último ejemplo:

```
result = pd.merge(left, right, on=["key1", "key2"], how="left")
result
```

```
result = pd.merge(left, right, on=["key1", "key2"], how="left")
result
```

	key1	key2	A	B	C	D
0	K0	K0	A0	B0	C0	D0
1	K0	K1	A1	B1	NaN	NaN
2	K1	K0	A2	B2	C1	D1
3	K1	K0	A2	B2	C2	D2
4	K2	K1	A3	B3	NaN	NaN

Figura 16. Función `merge()` con `left` en el parámetro `how`.

```
result = pd.merge(left, right, on=["key1", "key2"], how="right")
result
```

```
result = pd.merge(left, right, on=["key1", "key2"], how="right")
result
```

	key1	key2	A	B	C	D
0	K0	K0	A0	B0	C0	D0
1	K1	K0	A2	B2	C1	D1
2	K1	K0	A2	B2	C2	D2
3	K2	K0	NaN	NaN	C3	D3

Figura 17. Función `merge()` con `right` en el parámetro `how`.

```
result = pd.merge(left, right, on=["key1", "key2"], how="outer")
result
```

```
result = pd.merge(left, right, on=["key1", "key2"], how="outer")
result
```

	key1	key2	A	B	C	D
0	K0	K0	A0	B0	C0	D0
1	K0	K1	A1	B1	NaN	NaN
2	K1	K0	A2	B2	C1	D1
3	K1	K0	A2	B2	C2	D2
4	K2	K0	NaN	NaN	C3	D3
5	K2	K1	A3	B3	NaN	NaN

Figura 18. Función `merge()` con `outer` en el parámetro `how`.

```
result = pd.merge(left, right, on=["key1", "key2"], how="cross")
result
```

```
result = pd.merge(left, right, how="cross")
result
```

	key1_x	key2_x	A	B	key1_y	key2_y	C	D
0	K0	K0	A0	B0	K0	K0	C0	D0
1	K0	K0	A0	B0	K1	K0	C1	D1
2	K0	K0	A0	B0	K1	K0	C2	D2
3	K0	K0	A0	B0	K2	K0	C3	D3
4	K0	K1	A1	B1	K0	K0	C0	D0
5	K0	K1	A1	B1	K1	K0	C1	D1
6	K0	K1	A1	B1	K1	K0	C2	D2
7	K0	K1	A1	B1	K2	K0	C3	D3
8	K1	K0	A2	B2	K0	K0	C0	D0
9	K1	K0	A2	B2	K1	K0	C1	D1
10	K1	K0	A2	B2	K1	K0	C2	D2
11	K1	K0	A2	B2	K2	K0	C3	D3
12	K2	K1	A3	B3	K0	K0	C0	D0
13	K2	K1	A3	B3	K1	K0	C1	D1
14	K2	K1	A3	B3	K1	K0	C2	D2
15	K2	K1	A3	B3	K2	K0	C3	D3

Figura 19. Función `merge()` con `cross` en el parámetro `how`.

join()

La idea de la función `join` es **relacionar las filas de un DataFrame con las filas de otro, ya sea de acuerdo con el índice o a una columna clave**.

Esta función puede recibir varios parámetros, los cuales puedes consultar a detalle en la documentación oficial de pandas. Es muy parecida a la función `merge()` y también puede utilizarse para implementar operaciones de `JOIN`. Aquí revisaremos algunos ejemplos que ilustran sus usos más comunes.

Para empezar, definamos un par de DataFrames, uno llamado `df` y el otro llamado `other`:

```
df = pd.DataFrame({'key': ['K0', 'K1', 'K2', 'K3', 'K4', 'K5'],  
                  'A': ['A0', 'A1', 'A2', 'A3', 'A4', 'A5']})  
  
other = pd.DataFrame({'key': ['K0', 'K1', 'K2'],  
                     'B': ['B0', 'B1', 'B2']})
```

df

key

A

0

K0

A0

1

K1

A1

2

K2

A2

3

K3

A3

4

K4

A4

5

K5

A5

other

key

B

0

K0

B0

1

K1

B1

2

K2

B2

Figura 20. Definición de DataFrames `df` y `other`.

Puedes pedirle al DataFrame `df` que realice la operación `join` con el DataFrame `other`, pero como existen columnas con el mismo nombre, si solo ejecutamos:

```
python
CopiarEditar
df.join(other)
```

se generaría un error, ya que el DataFrame resultante tendría dos columnas con el mismo nombre. Por lo tanto, debemos especificar **sufijos** para evitar estas coincidencias en los nombres de las columnas.

Esto lo podemos hacer especificando valores en los **parámetros** `lsuffix` y `rsuffix`, quedando la instrucción de la siguiente manera:

```
python
CopiarEditar
df.join(other, lsuffix='_caller', rsuffix='_other')
```

```
df.join(other, lsuffix='_caller', rsuffix='_other')
```

	key_caller	A	key_other	B
0	K0	A0	K0	B0
1	K1	A1	K1	B1
2	K2	A2	K2	B2
3	K3	A3	NaN	NaN
4	K4	A4	NaN	NaN
5	K5	A5	NaN	NaN

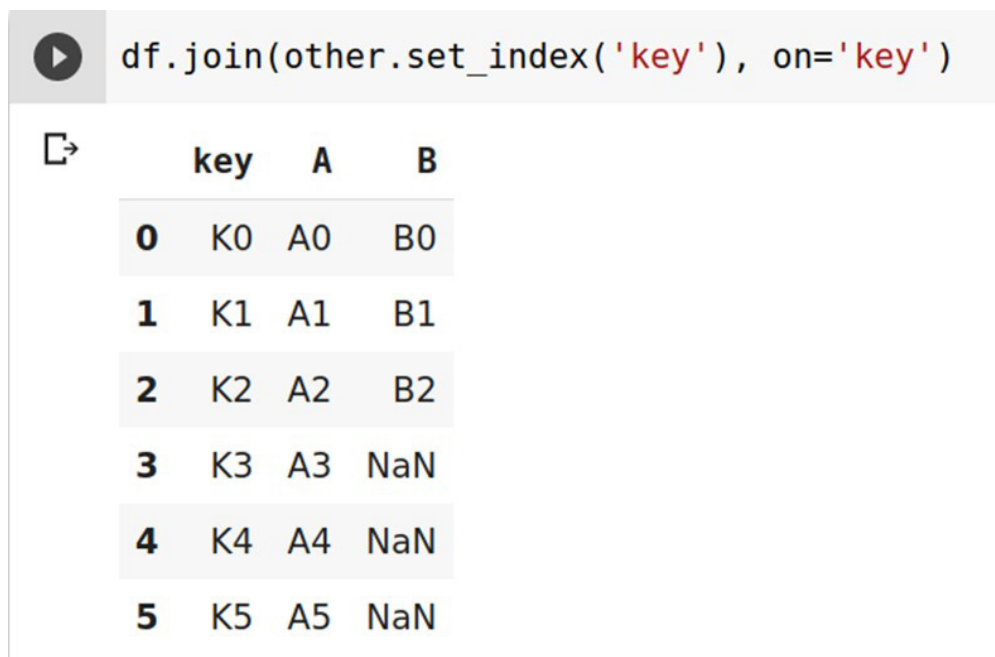
Figura 21. Uso de los parámetros `lsuffix` y `rsuffix`.

En este resultado podemos apreciar cómo se hace el `JOIN` usando el índice de ambos DataFrames, notando también que por *default* se ejecuta un “`left join`”, aunque también podemos utilizar el parámetro `how` para especificar qué tipo de `join` queremos (`inner`, `left`, `right` o `outer`).

De la misma forma, podemos realizar la operación de reunión (`JOIN`) utilizando cualquier columna. En este caso, usaremos la columna “**key**”. Sólo hay que tener en cuenta que, para usar cualquier columna que no sea el índice, es necesario especificar que se desea usar esa columna como índice.

Por ejemplo, para usar la columna “**key**” en ambos DataFrames para hacer la operación `JOIN`, la instrucción quedaría de la siguiente manera:

```
df.join(other.set_index('key'), on='key')
```



The screenshot shows a Jupyter Notebook cell with the following code and output:

```
df.join(other.set_index('key'), on='key')
```

The output is a DataFrame with 6 rows and 4 columns: **key**, **A**, and **B**. The rows are indexed from 0 to 5.

	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2
3	K3	A3	NaN
4	K4	A4	NaN
5	K5	A5	NaN

Figura 22. Operación `join` en la columna `key` de ambos DataFrames.

En esta instrucción, le pedimos al DataFrame `df` que ejecute la instrucción `join` con el DataFrame `other`, al cual le fijamos como índice la columna `key`, y finalmente, pedimos que la columna que usaremos como índice del DataFrame `df` sea `key`.

Para más información, consulta la documentación oficial incluida en la sección de Material adicional de este recurso.

Dado que `merge` y `join` son operaciones tan similares, entonces, ¿cuáles son sus diferencias?

- `merge` es una función más versátil, ya que puedes utilizarla llamando a la función del módulo pandas (`pd.merge(df1, df2, ...)`), o como una función de un DataFrame (`df1.merge(df2, ...)`), mientras que `join` es una función del DataFrame (`df1.join(df2, ...)`)
- Cuando utilizamos `join`, internamente se hace una llamada a `merge`, pero de una manera optimizada. `join` nos proporciona una sintaxis que puede ser más fácil de entender.
- `join` trabaja por defecto con un `left join`, mientras que `merge` lo hace por *default* con un `inner join`.
- `join` trabaja con índices, mientras que en `merge` le podemos especificar las columnas con las que queremos que trabaje.

Retomando la función `concat` revisada previamente: `concat()` es una función de pandas que llamamos de la forma `pd.concat`, porque es una función de la librería pandas y nos sirve para concatenar información de DataFrames tanto como filas, como columnas, es decir, “pegar” DataFrames. Ya sea las filas de un DataFrame debajo de las filas de otro (concatenación vertical, forma por defecto, `axis=0`), o para “pegar” las columnas de un DataFrame “al lado” de las columnas de otro (concatenación horizontal, usando `axis=1`), mientras que `merge` y `join` nos sirven para relacionar información proveniente de dos DataFrames.

Seguramente en la práctica constante, encontrarás escenarios que se resuelvan con más de una opción de estas operaciones.

Antes de terminar...

Te hemos presentado funciones para conjuntar información proveniente de diferentes fuentes, y te dimos ejemplos que esperamos que te sigan ayudando a tener un mejor entendimiento de dichas funciones.

Continuando con el manejo de la información que llevas en tu trabajo, reflexiona lo siguiente:

- ¿Cómo podrías utilizar estas funciones para crear DataFrames que representen información de tu lugar de trabajo en donde se tenga un DataFrame con información de varias fuentes para poder hacer análisis a esa información?
- ¿Cuál o cuáles de las funciones vistas crees que te puedan servir más para trabajar con la información con la que trabajas día a día?



| Material adicional

Pandas. (s.f.). *Merge, join, concatenate and compare*. [Fusionar, unir, concatenar y comparar]. Pandas. Recuperado el 15 de abril de 2025 de: https://pandas.pydata.org/pandas-docs/stable/user_guide/merging.html

Pandas. (s.f.). *pandas.DataFrame.join*. Pandas. Recuperado el 15 de abril de 2025 de: <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.join.html>

Stratis, K. (s.f.). *Combining Data in pandas With merge(), .join(), and concat()*. [Combinando datos en pandas con merge(), .join() y concat()]. Real Python. Recuperado el 15 de abril de 2025 de: <https://realpython.com/pandas-merge-join-and-concat/>

Nota: Los enlaces a las páginas web incluidos en este Material adicional fueron consultados y verificados por última vez el 15 de abril del 2025 y su disponibilidad depende del autor.

| Glosario

`concat()` : Función de pandas utilizada para concatenar (unir) objetos tipo DataFrame o Series. Permite unir datos de forma vertical u horizontal, dependiendo del parámetro `axis`.

Concatenación horizontal: Forma de unir DataFrames uno al lado del otro (por columnas), lo cual se logra con `concat()` usando el parámetro `axis=1`.

Concatenación vertical: Forma de unir DataFrames uno debajo del otro (por filas), lo cual es el comportamiento predeterminado de `concat()` cuando `axis=0`.

DataFrame: Estructura de datos bidimensional de la biblioteca pandas, que se asemeja a una tabla y permite almacenar y manipular datos etiquetados en filas y columnas.

`how`: Parámetro de `merge()` y `join()` que define el tipo de combinación que se realizará: "inner", "left", "right", "outer" o "cross".

`join()` : Método simplificado del DataFrame para unir información de otro DataFrame usando índices o una columna clave.

`merge()` : Función de pandas para combinar dos DataFrames en uno sólo, relacionando columnas comunes.

NaN: Acrónimo de "Not a Number". Representa un valor faltante o inexistente en un DataFrame, comúnmente utilizado en operaciones de unión cuando no hay coincidencia.

`reset_index()` : Función que reinicia los índices de un DataFrame.

`set_axis()` : Función que permite renombrar las etiquetas de filas o columnas en un DataFrame.

I Bibliografía

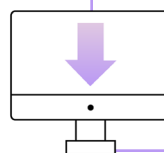
Pandas (s.f.) *pandas.concat*. Pandas. Recuperado el 25 de abril de 2025 de: <https://pandas.pydata.org/docs/reference/api/pandas.concat.html>

Pandas (s.f.) *pandas.DataFrame.join*. Pandas. Recuperado el 25 de abril de 2025 de: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.join.html>

Pandas (s.f.) *Merge, join, concatenate and compare*. [Unir, juntar, concatenar y comparar] Pandas. Recuperado el 25 de abril de 2025 de: https://pandas.pydata.org/docs/user_guide/merging.html

Pandas (s.f.) *pandas.merge*. Pandas. Recuperado el 25 de abril de 2025 de: <https://pandas.pydata.org/docs/reference/api/pandas.merge.html>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 25 de abril del 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.